Verify, Probe, Fall Back: A Safe Selection Layer for Rule-Based and LLM-Proposed Query Rewrites

Yaoning Liu
Beijing University of Posts and Telecommunications, Beijing, China
1652141467@qq.com ORCID: 0009-0003-2754-9159

***Abstract*—Rule engines and large language models (LLMs) both rewrite a slow Structured Query Language (SQL) statement into an equivalent statement that is meant to run faster, but a rewrite that returns different rows or runs slower than the original is worse than no rewrite. We address one concrete setting: recurring analytical statements, such as scheduled reports, dashboard queries, and extract, transform, load jobs, on a PostgreSQL server whose administrator can spend a bounded tuning window on a statement because the statement runs many more times afterwards. For this setting we present VeriProbe, a selection layer that accepts candidate rewrites from any source, admits a candidate only when an equivalence gate accepts it against the original statement, probes at most two admitted candidates on the target database, and returns the original statement unless a fully timed candidate is faster. We evaluate VeriProbe with Apache Calcite rule rewrites, replayed published rewrites of R-Bot and QUITE, frozen LLM proposals, and planner hints on the Transaction Processing Performance Council benchmark H (TPC-H), the decision support benchmark DSB, and the Calcite test workload. Executions are cut off at different times, so we report every latency as an interval instead of replacing a cut off run by a constant, and we time the returned statements again in one session with one cutoff. On 131 held-out statements VeriProbe improves 23 statements and lowers mean latency by 18.8 %; the session finds 22 of 23 returned rewrites faster than the original and 1 slower. VeriProbe is faster than a replay of the QUITE rewrites. It does not outperform a replay of the R-Bot rewrites: it keeps 6 originals that are cut off although a replayed rewrite in its pool finishes, and it returns 11 faster rewrites where neither replay does. 55 of 62 selected rewrites rest on bounded testing, not on proof.**

***Index Terms*—query rewriting, query optimization, large language models, query equivalence, PostgreSQL**

I. Introduction

Query rewriting transforms a Structured Query Language (SQL) statement into an equivalent statement that a database management system (DBMS) executes faster. Classical rewriters apply a fixed set of transformation rules inside the optimizer [1]–[3]. Recent systems move the rewriter outside the DBMS. One line of work searches over sequences of Apache Calcite rules [4]–[6]; R-Bot lets a large language model (LLM) choose and order such rules with retrieved rewrite evidence [7]. A second line lets the LLM write the new statement itself, as in GenRewrite and QUITE [8], [9].

Application scenario. We consider a database administrator (DBA) who operates a PostgreSQL server for decision support. The workload contains recurring statements: scheduled reports, dashboard queries, and the transformation steps of extract, transform, load jobs. Each such statement runs again every hour or every night with the same text. The DBA is willing to spend a bounded tuning window on a slow statement once, because every later execution profits. The DBA accepts a rewrite only under two conditions: it returns the same rows as the original, and it is not slower than the original on this server. Fig. 1 shows this setting.

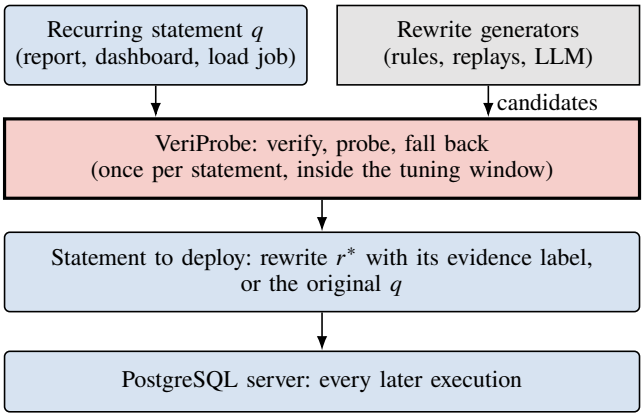


Fig. 1. Application scenario. The layer runs once per recurring statement; the statement that it returns serves every later execution.

Gap. Rewriters outside the DBMS do not meet these two conditions by themselves. First, an LLM can produce a statement that is not equivalent, and a rule sequence replayed on a different schema can fail in the same way. Second, a rewriter usually ranks its candidates by the planner's cost estimate, and estimates are unreliable for complex statements [10]. Recent tools add verification and execution feedback to a single generator: a tuning tool for Microsoft SQL Server verifies LLM rewrites with the optimizer and spends a time budget on execution [11], and ReSequel verifies LLM rewrites on sampled data and ranks them by performance [12]. Two questions remain open. (i) Each system selects within its own candidate space, so it is unknown what a DBA gains from one verified selection step placed on top of several existing rewriters at once. (ii) Rewriters are compared under time limits, and a run that exceeds its limit is commonly charged a fixed penalty. When the limit differs between the compared arms, the penalty decides the comparison, not the measurement. Our logs show this effect directly: for one replayed arm, the charged value alone moves the result from a reduction of mean latency by 3.1 % to an increase by 24.1 %.

Approach. We present VeriProbe, a selection layer between

rewrite generators and the DBMS. VeriProbe does not generate rewrites. It takes a pool of candidates from any source, admits only candidates that pass an equivalence gate against the original statement, orders the admitted candidates by planner estimates without executing them, probes at most two of them with one bounded execution each, times the best probe fully, and returns the original statement unless that candidate is faster (Fig. 2). Every returned rewrite carries a label that states whether its equivalence evidence is a proof or bounded testing. We evaluate VeriProbe on the Transaction Processing Performance Council benchmark H (TPC-H), the decision support benchmark DSB, and the Calcite test workload.

Findings. The evaluation supports three statements and contradicts the expectation that the layer outperforms each of its generators.

- On a development cohort of 30 expensive statements, VeriProbe improves 22 statements and lowers mean latency from 168.8 s to 62.3 s. The candidate pool of this cohort is tuned on the cohort, and its timings are assembled from several campaigns, so we treat it as an upper reference.
- On 131 held-out statements, timed live, VeriProbe improves 23 statements (17.6 %) and lowers mean latency by 18.8 %. A validation session with one cutoff finds 22 of 23 returned rewrites faster than the original.
- VeriProbe is robustly faster than a replay of the published QUITE rewrites on held-out statements, and robustly faster than both replays on skewed data.
- VeriProbe does not outperform a replay of the published R-Bot rewrites on held-out statements. In the validation session the mean latency of the replay is [74.8, 82.2] s on the 42 held-out pairs for which an arm returns a rewrite, against [94.9, 99.5] s for VeriProbe. VeriProbe keeps 6 originals that are cut off although a replayed rewrite in its pool finishes; in turn it returns 11 faster rewrites where neither replay does.

Contributions.

- A selection layer for the scenario above that separates generation from admission and selection. Its inputs are heterogeneous: rule rewrites, replayed outputs of other systems, LLM proposals, and planner hints (Sections III and IV).
- An equivalence gate that fails closed and labels its evidence as syntactic, formal, or bounded (Section V).
- A budgeted selection procedure with an explicit bound on the executions it adds on the target database, and a fallback that is anchored on the original statement (Section VI).
- An interval protocol for reporting rewrite latency under unequal cutoffs, with paired outcome classes that separate measured differences from undetermined ones, and a validation session that resolves them with one cutoff (Sections VII and VIII-G).
- An evaluation on 211 pairs of database and statement that reports where the layer helps, where it does not, and which of its results hold only by construction (Sections VIII and IX).

The paper does not propose a new rewrite generator, and the layer combines known ingredients. Its novelty is the scenario specific combination and the measurement of what this combination adds to existing rewriters.

II. Preliminaries and Problem Statement

Statements and equivalence. A statement q is a single read query over a schema Σ. For a database instance D of Σ, $q(D)$ denotes the result: a bag of rows, or a sequence of rows when q has an outermost ordering. Two statements q and p are equivalent, written $q \equiv p$, when $q(D') = p(D')$ for every instance D' of Σ. Deciding equivalence is undecidable for full SQL, so practical tools either prove it for a fragment [13]–[17] or check it on a bounded set of instances [18].

Evidence classes. We distinguish three classes of evidence for $q \equiv p$: *syntactic* (the two statements have the same tokens up to formatting and planner hints), *formal* (a prover establishes equivalence for all instances), and *bounded* (the results agree on a fixed finite set of instances, which can miss a difference that only other instances expose).

Latency. The latency $\ell_D(q)$ is the execution time that the server reports for q on D. We measure it as the median of three runs after one warmup run. Every run has a cutoff; a run that reaches its cutoff yields only a lower bound on the latency.

Problem. Given a statement q, a target database D, and a pool $P(q)$ of candidate rewrites from arbitrary generators, the ideal output is

$$\begin{cases} V(q) = \{\, p \in P(q) : G(q,p) = 1 \,\}, \\ r^* = \underset{r \in \{q\} \cup V(q)}{\arg\min}\ \ell_D(r), \end{cases} \tag{1}$$

where q is the original statement, $P(q)$ is the candidate pool, $G(q,p) \in \{0,1\}$ is the decision of the equivalence gate for the pair (q,p), $V(q)$ is the set of admitted candidates, r^* is the statement to return, D is the target database, and $\ell_D(r)$ is the latency of a statement r on D. The set $\{q\} \cup V(q)$ always contains q, so the output is never slower than the original when latencies are known exactly. Latencies are known only through execution, and each execution on D costs time inside the tuning window. The problem is therefore to approximate (1) with a bounded number of executions on D.

III. System Overview

Fig. 2 shows the five steps of VeriProbe and the data that flows between them.

Generators produce the pool $P(q)$ from the statement and the schema (step 1, Section IV). The gate checks each candidate against the original statement on a small verification database and on five miniature databases, and it labels each admitted candidate with its evidence (step 2, Section V). VeriProbe then orders the admitted candidates by planner estimates and keeps the first M (step 3), probes at most two of them once on the target database (step 4), and times the

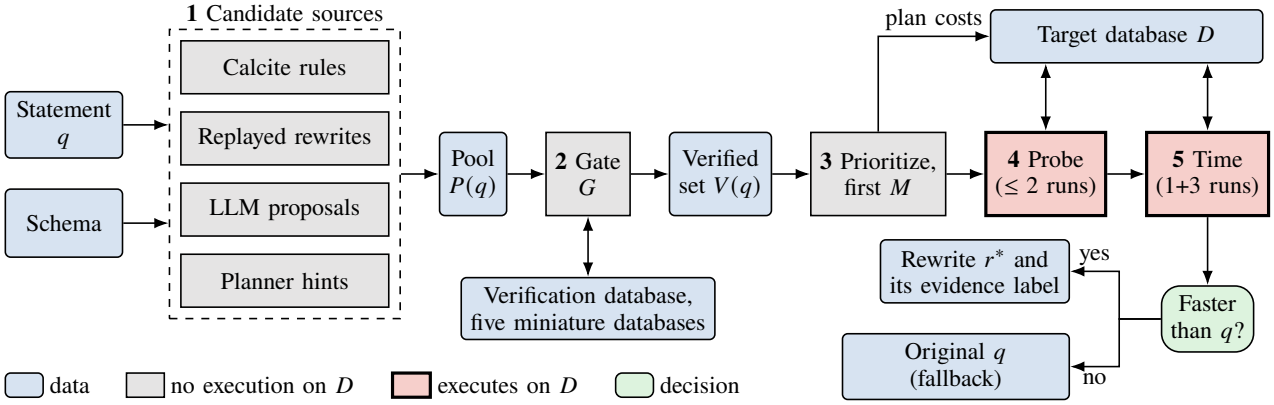


Fig. 2. Data flow of VeriProbe for one statement. Steps 1 to 3 never execute a candidate on the target database; steps 4 and 5 execute at most two probes and one full timing.

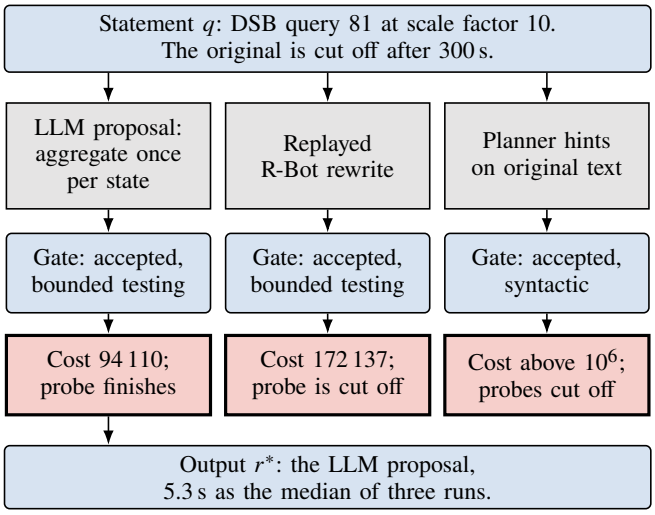


Fig. 3. Data flow for DSB query 81 at scale factor 10. Four admitted candidates that combine a rewrite with a planner hint are omitted; their probes are cut off or fail.

fastest probe with the same protocol as the original, returning it only when it is faster (step 5, Section VI).

Running example. Fig. 3 follows query 81 of DSB at scale factor 10, whose original run does not finish within 300 s. The statement compares the total returns of every customer with 1.2 times the average total of the customer's state; the average is a correlated subquery. The gate admits all 8 candidates of the pool. An LLM proposal computes the averages per state once in a materialized common table expression (planner cost 94 110); the replayed R-Bot rewrite has planner cost 172 137. Only the probe of the LLM proposal finishes; its full timing gives 5.3 s, and VeriProbe returns it with the label *bounded*.

IV. Candidate Sources

VeriProbe distinguishes two classes of candidates by how they are produced.

Class A: rule rewrites. A class A candidate is the output of Apache Calcite rules [4] applied to the original statement. We obtain these candidates in three ways: (i) the rewritten statements of R-Bot that ship with the public QUITE repository, (ii) rule sequences from the released R-Bot logs, which we apply again through the Calcite executor that R-Bot ships, and (iii) single Calcite rules whose pattern matches the statement, used only when (i) and (ii) yield nothing and only in campaigns that include Calcite statements.

Class B: free rewrites and planner hints. A class B candidate is a statement that no fixed rule set produces. We use three kinds. (i) *LLM proposals.* A language model receives the statement, the schema, and a plan from a small development database, and proposes rewritten statements. We generate all proposals once, before any timing on the target databases, and freeze them in a file: 666 proposals for 439 distinct statements. The generation inspects neither published rewrites nor latencies on the target databases. (ii) *Replayed QUITE rewrites.* We take the rewritten statements that the QUITE authors publish [9]. (iii) *Planner hints.* A candidate keeps the text of a statement and adds a comment that the pg_hint_plan extension [19] interprets, for example to disable nested loop joins for this statement. A hint changes the plan and never the result; steering the optimizer with hints is an established technique [20].

Composition. A candidate that is produced from another candidate has depth 2 or more. The gate always compares a candidate with the original statement, never with its parent, so an error cannot accumulate along a chain.

Pool construction. Fig. 4 shows how the pool of one statement is built. The runner first collects the frozen LLM proposals and the published rewrites that are stored for the text of q. If no rule rewrite is among them, it applies up to eight matching Calcite rules, one rule at a time. The runner also normalizes count aggregates over columns that are declared not null, which adds one more candidate where it applies. The runner then treats every new candidate like q: it looks up proposals for the text of the candidate, and for the 12 statements of the development cohort that Section VII-A describes it adds planner hints. Expansion stops when no new candidate appears or when depth five is reached. Finally the runner

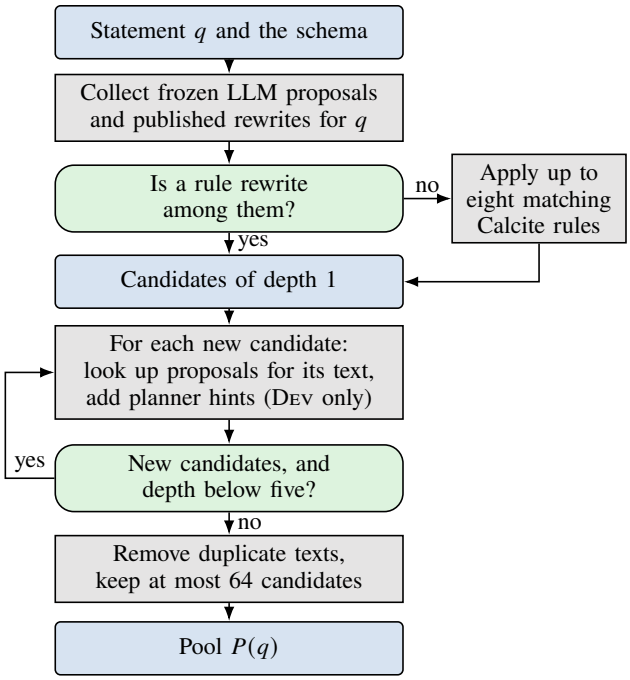


Fig. 4. Construction of the candidate pool $P(q)$ for one statement. Planner hints are added only for statements of the development cohort.

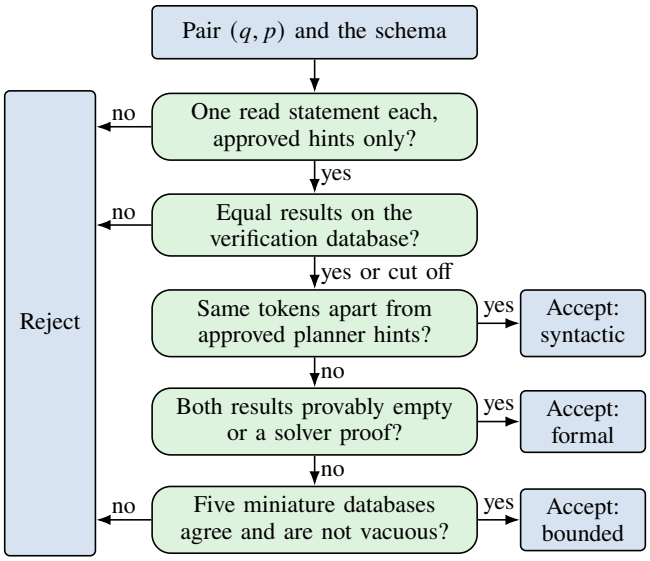


Fig. 5. Decision flow of the equivalence gate. A candidate is admitted only through one of the three accepting exits, and the exit determines its evidence label.

removes duplicate texts and keeps at most 64 candidates. In our campaigns the largest pool holds 26 candidates, no admitted candidate is deeper than 2, and 10 statements, all of them TPC-H statements, have an empty pool.

V. Equivalence Gate

The gate decides $G(q, p)$ for a pair of statements and returns an evidence label. It fails closed: an unsupported construct, an error, or an inconclusive result rejects the candidate. Fig. 5 shows the decision flow.

Admissible input. Each side must parse as exactly one read statement. A planner hint is admissible only when it belongs to an approved list of plan directives: join methods, scan methods, join order, and planner switches. A comment that can change results is rejected.

Execution check. The gate executes both statements on a verification database in one snapshot and compares the results as bags, or as sequences when the statement orders its output. The verification database has the schema of the target database and little data: TPC-H at scale factor 0.01, DSB at scale factor 1, and 10,000 rows per main table for the Calcite schema. The gate first confirms that both databases have the same tables, columns, types, keys, and collations. The comparison is positional and tolerates a relative numeric difference of 10^{-5}. Unequal results reject the candidate. If the comparison exceeds its time limit, the gate continues with the remaining checks, because a slow original is not evidence against equivalence.

Syntactic exit. If the two statements have the same token sequence apart from approved planner hints, the gate accepts with the label *syntactic*.

Formal exit. Two provers can accept. The first is SQL-Solver [16], which the gate calls only for a restricted fragment: select, project, join, set operations, and Boolean predicates over integer and text columns, with all key and null constraints removed from the schema that the prover receives. The second is a structural check for one pattern: both statements provably return no row, for example because a filter requires a window count to be null. A prover verdict of not equivalent rejects the candidate.

Bounded exit. Otherwise the gate runs both statements on five miniature databases that it builds from the schema and from the literals of the statement: an empty database, one with null values, one with duplicate rows, one with skewed values, and one with unmatched join keys. The gate accepts with the label *bounded* when all five results agree and the evidence is not vacuous: at least one miniature must return rows, and the five results must not all be identical.

What the gate does not establish. The labels *syntactic* and *formal* cover all instances. The label *bounded* covers six instances. The gate never compares results on the target database; Section VIII-F reports how often each label occurs.

VI. Budgeted Selection with Fallback

The selection procedure approximates (1) with few executions on the target database.

Prioritization. Each admitted candidate $p \in V(q)$ receives the key

$$\pi(p) = \big(d(p),\ \mathbf{1}[\kappa(p) \geq \kappa(q)],\ b(p),\ \kappa(p),\ i(p)\big), \qquad (2)$$

where $\pi(p)$ is the key, $d(p)$ is the depth of p, $\kappa(\cdot)$ is the total cost that the planner of the target database estimates for a statement, $\mathbf{1}[\cdot]$ equals 1 when its argument holds and 0 otherwise, $b(p)$ equals 0 when the action that produces p or the estimated plan of p involves a join, a nested loop, a

subplan, a materialization, or a decorrelation, and 1 otherwise, and $i(p)$ is the position of p in the order of generation. VeriProbe sorts $V(q)$ by π in ascending lexicographic order and keeps the first M candidates as the prefix $V_M(q)$. The order uses only planner estimates and is fixed before any candidate executes.

Probes. A probe is one execution of a candidate on the target database, without a warmup run, with the cutoff

$$c_{\mathrm{p}}(q) = \min\bigl(\ell_0(q),\, \bar{c}\bigr), \tag{3}$$

where $c_{\mathrm{p}}(q)$ is the probe cutoff, $\ell_0(q)$ is the measured latency of the original statement, set to the timeout $T_{\max}$ when the original is cut off, and $\bar{c}$ is a fixed cap. We use $T_{\max} = 300\,\mathrm{s}$ and $\bar{c} = 45\,\mathrm{s}$; for five statements of the development cohort with long originals, $\bar{c} = 120\,\mathrm{s}$. VeriProbe walks through $V_M(q)$ in priority order and probes at most two candidates. It reuses a result without a new execution in two cases: a timing of the same statement text exists in the measurement cache, or the candidate has the same estimated plan as a statement that is already timed. After one probe improves on the original by more than 15 %, VeriProbe skips candidates that carry no planner hint and whose planner cost is not below that of the original.

Timing and decision. Let $\tilde{\ell}(p)$ denote the probe time of p and $\ell(p)$ the median of three timed runs of p under the cutoff

$$c_{\mathrm{f}}(q) = \min\bigl(T_{\max},\, \max(60\,\mathrm{s},\, 1.05\,\ell_0(q))\bigr), \tag{4}$$

where $c_{\mathrm{f}}(q)$ is the cutoff of the timed runs, and $T_{\max}$ and $\ell_0(q)$ are as in (3). The output of VeriProbe is

$$\begin{cases} S(q) = \{\, p \in V_M(q) : \tilde{\ell}(p) < \ell_0(q) \,\}, \\ p^* = \arg\min\limits_{p \in S(q)} \tilde{\ell}(p), \\ r^* = \begin{cases} p^*, & S(q) \neq \emptyset \text{ and } \ell(p^*) < \ell_0(q), \\ q, & \text{otherwise}, \end{cases} \end{cases} \tag{5}$$

where $S(q)$ is the set of probed candidates that finish faster than the original, p^* is the fastest probe, and r^* is the returned statement. When a second member of $S(q)$ already has a full timing in the cache, VeriProbe compares both full timings and keeps the faster one. A probe that takes at least 10 s counts as the first of the three timed runs, so that a long candidate executes three times in total. Fig. 6 shows the control flow of the procedure, and Fig. 7 shows the executions that it places on the target database. In Fig. 6 the two steps with a heavy outline are the only ones that execute a candidate on D.

Cost bound. Beyond the timing of the original statement, the executions that VeriProbe adds on D take at most

$$C(q) \le 2\,c_{\mathrm{p}}(q) + 4\,\ell(p^*), \tag{6}$$

where $C(q)$ is the time of these executions, $c_{\mathrm{p}}(q)$ is the probe cutoff of (3), and $\ell(p^*)$ is the latency of the fully timed candidate, counted four times for one warmup and three runs. A rewrite that saves $\ell_0(q) - \ell(r^*)$ per execution recovers this cost after

$$n(q) = \frac{C(q)}{\ell_0(q) - \ell(r^*)} \tag{7}$$

executions, where $n(q)$ is this number of executions, $C(q)$ is the bound of (6), $\ell_0(q)$ is the latency of the original, and $\ell(r^*)$ is the latency of the returned rewrite.

Properties. Two properties follow from (5) and do not depend on the quality of the candidates: the returned statement is the original or an admitted candidate, and it is not slower than the original in the three timed runs that VeriProbe compares. The second property holds for these runs; it is not a prediction for later executions (Section VIII-G).

VII. Evaluation Methodology

A. Setup

Server. All experiments run on one workstation with an Intel Core i5-12450H processor (8 cores), 31 GiB of main memory, and Ubuntu 24.04. The DBMS is PostgreSQL 14.13 with the pg_hint_plan extension, 256 MiB of shared buffers, 8 MiB of working memory per operation, parallel query execution disabled, and just-in-time compilation disabled. This configuration is small for the data sizes below, and it makes many original statements slow. We keep it fixed for all arms.

Data. We use TPC-H [21] at scale factors (SF) 10 and 50, DSB [22] at SF 10 and 50, and the schema of the Calcite test suite filled with 40 million rows per main table by the generator of SlabCity [23], once with uniform and once with Zipf distributed values.

Statements. The public repositories of R-Bot and QUITE contain 441 statements for these schemas: 107 for TPC-H, 232 for DSB, and 102 for Calcite. We evaluate all 107 TPC-H statements, the 44 Calcite statements of R-Bot, and 10 DSB statements. The remaining 222 DSB statements and the 58 Calcite statements of QUITE are not evaluated (Section IX).

Cohorts. Table I lists the five cohorts. Dev contains 10 R-Bot statements each for TPC-H at SF 10, DSB at SF 10, and Calcite with uniform data. They are chosen before any timing by descending planner cost, one statement per query family, below a cost ceiling that excludes plans the planner rates as pathological. The pool is developed on Dev: for 12 of its statements it contains planner hints that we add after inspecting their plans, for one statement it contains one rewrite that we write manually, and it contains 32 candidates collected in earlier runs on these statements. The two held-out cohorts contain all other evaluated TPC-H and Calcite statements; the pool of a held-out statement contains no manually added candidate. Zipf runs the first 30 Calcite statements on skewed data, and SF 50 runs the 20 TPC-H and DSB statements of Dev on five times more data. Fig. 8 shows the order of these steps.

Arms. We compare four arms on every statement. *Original* is the unchanged statement. *VeriProbe* is the procedure of Fig. 6 with $M = 16$ on the full pool. *R-Bot replay* returns the admitted rule rewrite of Section IV with the lowest planner cost, provided that this cost is below the cost of the original; it selects without probes and has no fallback. *QUITE replay* returns the first admitted published QUITE rewrite, also without probes and without fallback. Both replay arms return the original statement when they have no admitted

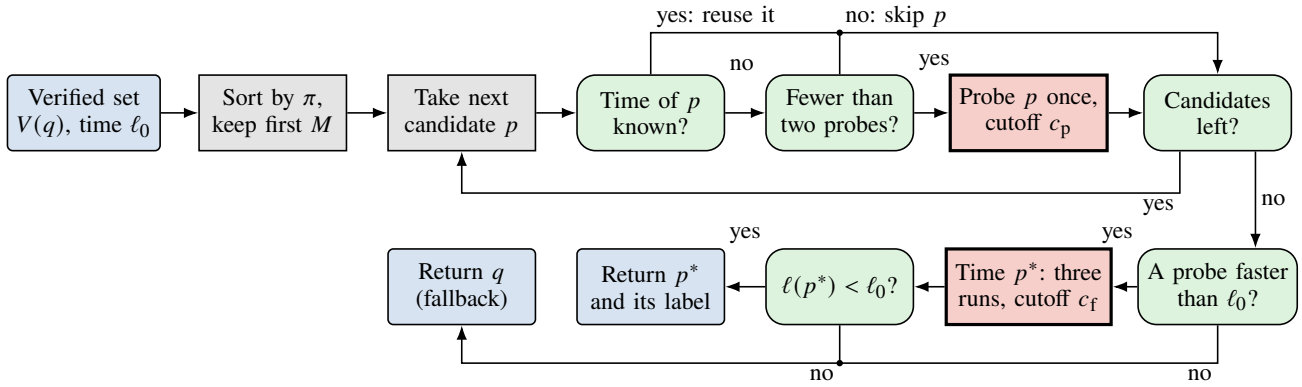


Fig. 6. Control flow of the selection procedure with budget M. Node styles are those of Fig. 2; steps with a heavy outline execute on the target database.

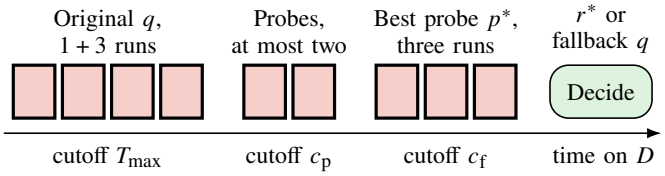


Fig. 7. Executions that VeriProbe places on the target database D for one statement. Each box is one run; the cutoffs are those of (3) and (4).

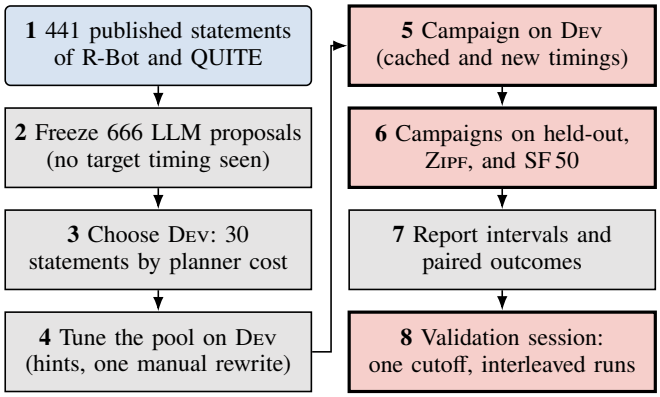


Fig. 8. Order of the experimental steps. Steps with a heavy outline execute statements on the target databases.

candidate. The replay arms are not runs of R-Bot and QUITE: their language model calls, retrieval, and feedback loops do not execute in our study. They measure what the published rewrites of these systems achieve when a DBA applies them on a different server after our gate.

Timing. Latency is the execution time that the PostgreSQL server reports when it executes a statement under its plan analysis command, with timing of individual plan nodes switched off. The original statement runs with the cutoff T_{max}; every other statement runs with the cutoff of (4). A replay arm does not time its rewrite directly. It first runs the rewrite once with a cutoff between 60 s and 120 s, or it reuses the probe of VeriProbe when VeriProbe has probed the same statement, and it times the rewrite fully only when this run finishes. At

TABLE I
Cohorts. Size: number of statements. Live: number of original statements that are timed in the same campaign as their candidates.

Cohort	Databases	Size	Live
Dev	TPC-H, DSB, Calcite	30	0
TPC-H held-out	TPC-H, SF 10	97	95
Calcite held-out	Calcite, uniform	34	34
Zipf	Calcite, Zipf	30	30
SF 50	TPC-H and DSB, SF 50	20	19

SF 50, an original whose run at SF 10 is cut off runs with a cutoff of 120 s. All LLM proposals come from one model, recorded in the artifact as gpt-6.1-sol. A measurement cache keyed by database and statement text avoids timing the same text twice. Code, manifests, frozen proposals, and raw logs are available at https://github.com/terraspacer/VeriProbe.

B. Interval Reporting under Unequal Cutoffs

A run that is cut off after c seconds shows only that the latency is at least c. Reports on query rewriting usually replace such a run by T_{max}. This is harmless when every arm has the cutoff T_{max}. It is not harmless in our logs, where the cutoff of a run ranges from the probe cutoff of (3) to 300 s: a rewrite that is stopped after 20 s and charged 300 s is charged 15 times the time for which it is observed. We therefore keep both ends of the interval. For an arm s and a statement q,

$$\left(\ell_s^L(q),\, \ell_s^U(q)\right) = \begin{cases} \left(m_s(q),\, m_s(q)\right), & \text{if the runs finish,} \\ \left(c_s(q),\, T_{max}\right), & \text{if the runs are cut off,} \end{cases} \tag{8}$$

where $\ell_s^L(q)$ and $\ell_s^U(q)$ are the lower and the upper end, $m_s(q)$ is the median of the three timed runs of the statement that arm s returns for q, capped at T_{max}, and $c_s(q)$ is the cutoff that applies to these runs. The log does not record $c_s(q)$ for every run. When a replay arm stops after its single first run, the log does not tell whether this run is a probe of VeriProbe; we set $c_s(q)$ to the probe cutoff of (3), the smaller of the two possible values. This concerns 28 of the 40 replayed rewrites that are cut off. For entries from earlier campaigns without a recorded

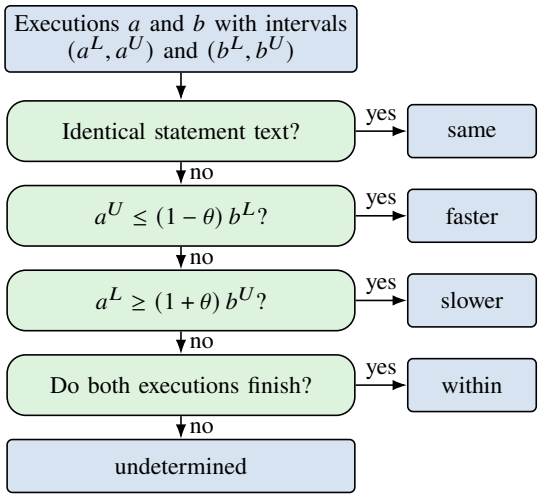


Fig. 9. Outcome class of an execution a against an execution b according to (10).

TABLE II
MEAN AND MEDIAN LATENCY IN SECONDS, AND OUTCOMES AGAINST THE ORIGINAL STATEMENT. INTERVALS FOLLOW (8). REWR.: STATEMENTS FOR WHICH THE ARM RETURNS A REWRITE; F: FASTER; S: SLOWER; U: UNDETERMINED.

Arm	Mean	Median	Rewr.	F	S	U
DEV, 30 statements, timings from the cache						
Original	168.8	141.0	–	–	–	–
R-Bot replay	[63.5, 138.2]	[34.7, 82.7]	15	7	0	8
QUITE replay	[99.4, 127.4]	[72.4, 95.7]	12	9	0	3
VeriProbe	62.3	41.0	22	22	0	0
TPC-H held-out, 97 statements, live timings						
Original	106.3	66.6	–	–	–	–
R-Bot replay	[89.7, 97.8]	[60.0, 64.7]	6	3	0	3
QUITE replay	[102.9, 131.8]	[58.4, 69.9]	11	0	0	11
VeriProbe	100.0	64.7	8	8	0	0
CALCITE held-out, 34 statements, live timings						
Original	98.5	17.8	–	–	–	–
R-Bot replay	[28.2, 52.0]	[2.8, 2.9]	12	9	0	3
QUITE replay	[74.7, 89.7]	9.2	5	2	1	2
VeriProbe	40.8	2.1	15	15	0	0

cutoff we set $c_s(q) = 20\,\text{s}$, the cap of the earliest campaign. For a cohort Q we report

$$\begin{cases} \bar{\ell}_s^x(Q) = \dfrac{1}{|Q|} \displaystyle\sum_{q \in Q} \ell_s^x(q), \\ R_s^x(Q) = 1 - \dfrac{\bar{\ell}_s^x(Q)}{\bar{\ell}_0^x(Q)}, \end{cases} \qquad x \in \{L, U\}, \tag{9}$$

where $\bar{\ell}_s^x(Q)$ is the mean latency of arm s under the lower ($x = L$) or upper ($x = U$) end of (8), $|Q|$ is the number of statements, the index 0 denotes the Original arm, and $R_s^x(Q)$ is the relative reduction of mean latency. We write a mean as one number when both ends coincide and as an interval otherwise.

For two executions a and b with intervals (a^L, a^U) and (b^L, b^U) we define the outcome of a against b as

$$o(a,b) = \begin{cases} \text{same}, & \text{identical statement text}, \\ \text{faster}, & a^U \le (1-\theta)\, b^L, \\ \text{slower}, & a^L \ge (1+\theta)\, b^U, \\ \text{within}, & \text{both finish, no bound holds}, \\ \text{undetermined}, & \text{otherwise}, \end{cases} \tag{10}$$

where $o(a,b)$ is the outcome class, $\theta = 0.1$ is a tolerance for timing noise, and the cases apply in the listed order, which Fig. 9 shows as a flowchart.

We call a statement *improved* by an arm when the outcome of the arm against Original is *faster*. We call a comparison *robust* when it holds under the end of each interval that is least favorable to it.

VIII. RESULTS

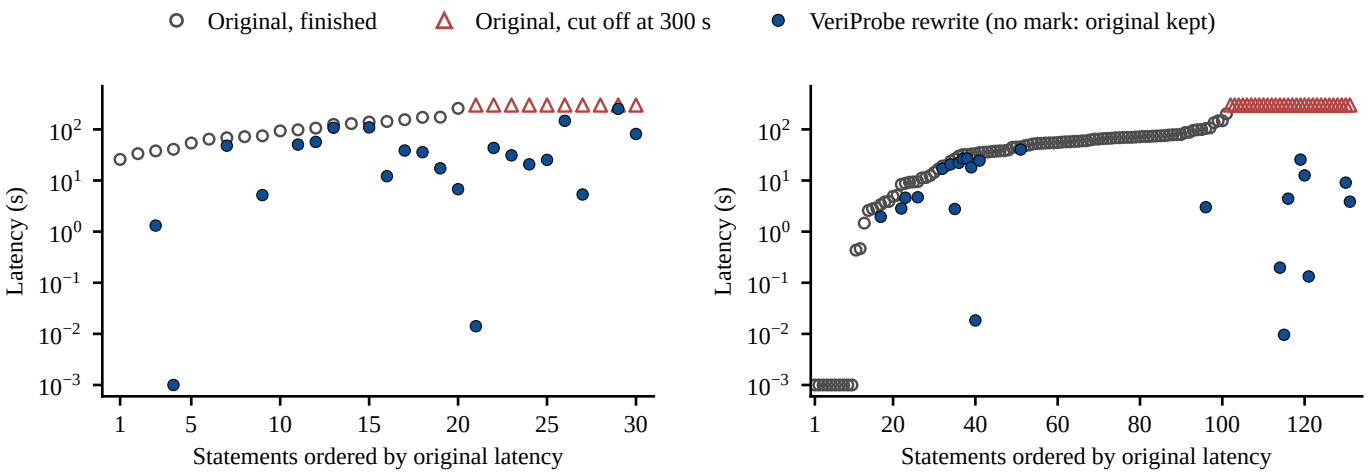
The experiments proceed in layers: whether the layer helps where its pool is tuned (Section VIII-A) and where it is not (Section VIII-B), how the result changes with the data (Section VIII-C), what the budget, the policy, and the sources contribute (Sections VIII-D and VIII-E), which evidence and cost stand behind the returned rewrites (Section VIII-F), and whether the timings hold in a session with one cutoff (Section VIII-G).

A. Development Cohort

Table II and Fig. 10(a) show the result on DEV. In Fig. 10 a circle marks an original that finishes, a triangle an original that is cut off, and a filled dot the rewrite that VeriProbe returns; a statement without a dot keeps its original. VeriProbe returns a rewrite for 22 of 30 statements and all of them are improvements; mean latency falls from 168.8 s to 62.3 s, a reduction of 63.1 %. The gain concentrates on statements that do not finish: 10 originals are cut off at 300 s, and VeriProbe returns a finishing rewrite for 9 of them.

Three facts bound what this cohort shows. First, the pool is tuned on it. 6 of the 22 improvements fall on the 12 statements with manually added hints, and 16 fall on the other 18 statements. Second, none of its original timings is live. They come from earlier campaigns through the measurement cache, and timings drift between campaigns on this shared workstation: one TPC-H statement takes 221.0 s in a run of one campaign and 54.1 s in another. Third, the earlier campaigns stop replayed rewrites early. 11 replayed rewrites are cut off, with a logged cutoff between 20 s and 21 s where the log records one, while cached timings of other candidates come from runs with cutoffs of up to 300 s.

The intervals make the third fact visible: the mean of R-Bot replay lies in [63.5, 138.2] s, and only its lower end is close to the 62.3 s of VeriProbe. In the paired view of Fig. 11, VeriProbe is faster than R-Bot replay on 14 statements and returns the same statement on 10; 6 comparisons are undetermined. Against QUITE replay, whose mean lies in [99.4, 127.4] s, VeriProbe is robustly faster.



(a) Dev, 30 statements

(b) Held-out, 131 statements

Fig. 10. Latency per statement: (a) development cohort; (b) held-out statements. Latencies below 1 ms are drawn at 1 ms.

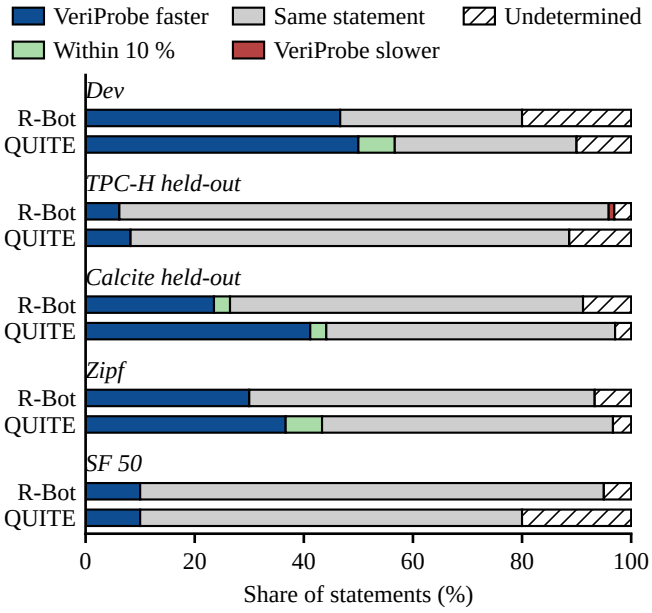


Fig. 11. Outcome of VeriProbe against each replay arm per cohort, as defined in (10). Each bar covers all statements of the cohort.

B. Held-Out Statements

The held-out cohorts are the main test: their pool contains no manually added candidate, and 129 of their 131 originals are timed in the same campaign as their candidates.

Against the original. VeriProbe improves 23 of 131 statements (17.6 %) and lowers mean latency from 104.3 s to 84.6 s (18.8 %). On Calcite held-out, 15 of 34 statements improve and the mean falls by 58.6 %; these statements are written to trigger specific rewrite rules, and 6 originals that do not finish receive a finishing rewrite. On TPC-H held-out, 8 of 97 statements improve and the mean falls by 5.9 %. For 89 TPC-H statements VeriProbe returns the original, and Fig. 10(b) shows that the originals that are cut off stay cut off with few exceptions. The pool explains this: the gate admits only 65 of 203 TPC-H candidates, and the probe cutoff of 45 s excludes every candidate that needs longer, although many originals run for more than 60 s.

Against the replay arms. The logs decide few comparisons of a replayed rewrite with its original. Of the 16 QUITE rewrites that the gate admits, the logs show 2 faster than the original, 1 slower, and 13 undetermined; of the 18 admitted R-Bot rewrites they show 12 faster, 0 slower, and 6 undetermined. Most undetermined rewrites are never timed, because the arm reuses a probe that is cut off. VeriProbe is robustly faster than QUITE replay: its mean of 84.6 s is below the lower end of [95.6, 120.9] s.

The comparison with R-Bot replay does not favor VeriProbe. The mean of R-Bot replay lies in [73.7, 85.9] s, which contains the mean of VeriProbe; on TPC-H held-out the replay is robustly better, [89.7, 97.8] s against 100.0 s. One statement decides this in the logs. Its original is cut off at 300 s and its replayed R-Bot rewrite takes 68.9 s. This rewrite is in the pool of VeriProbe and passes the gate, but VeriProbe never probes it: two LLM proposals with lower planner cost use both probes, both are cut off, and VeriProbe returns the original. Section VIII-G shows that this is not a single case.

All cohorts. Fig. 12 extends the outcome columns of Table II to all five cohorts: for every arm it shows the share of statements in each outcome class of (10). Only the replay arms have slower and undetermined shares; VeriProbe has none because of its fallback.

C. Skew and Scale

Fig. 13 repeats two sets of statements on a second database.

Skew. On Zipf data all timings are live. VeriProbe improves 14 of 30 statements and lowers the mean from 133.9 s to 55.9 s (58.2 %), close to the 64.6 % on the same statements with uniform data. The improved statements overlap but are not identical: 11 improve on both databases, 4 only on uniform

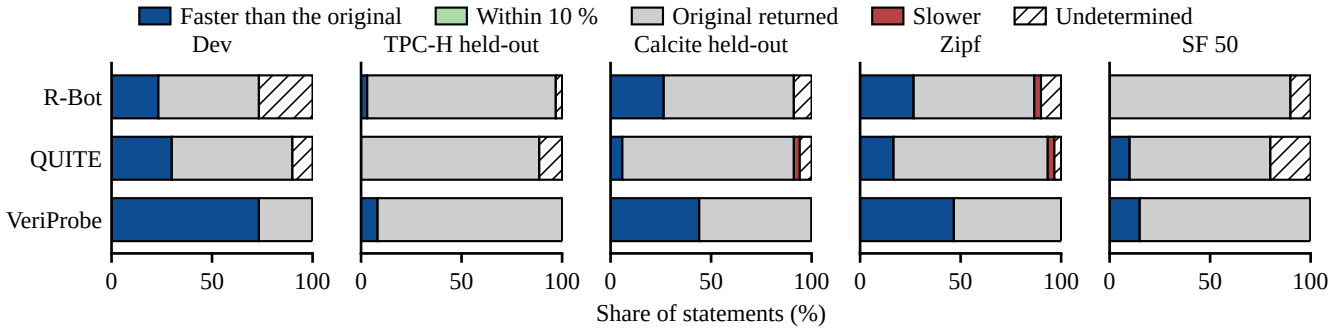


Fig. 12. Outcome of each arm against the original statement per cohort, as defined in (10).

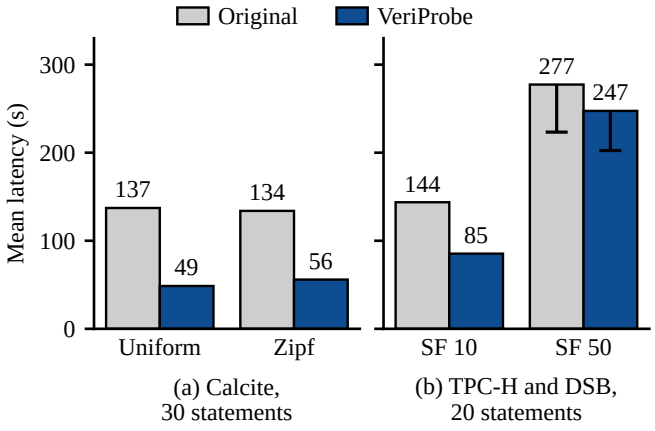


Fig. 13. Mean latency of the same statements on two databases: (a) uniform and Zipf data; (b) scale factors 10 and 50. A bar and the number above it show the upper end of (8); a whisker reaches down to the lower end where the two differ.

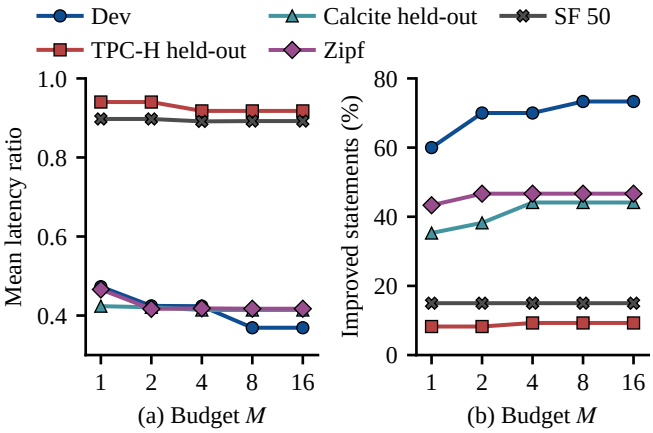


Fig. 14. Effect of the budget M: (a) mean latency of VeriProbe divided by that of Original, under the upper end of (8); (b) share of improved statements.

data, and 3 only on Zipf data. A rewrite that helps on one distribution is thus not a safe choice on another, which is the reason for selecting on the target database. On this cohort VeriProbe is robustly faster than both replay arms: their means lie in [74.4, 101.3] s and [100.8, 109.3] s.

Scale. At SF 50 the experiment is dominated by cutoffs: 16 of 20 originals do not finish, against 5 at SF 10, and some are stopped after 120 s, so that even the mean of Original is an interval, [223.4, 277.4] s. VeriProbe improves 3 statements, against 12 of the same statements at SF 10, and its mean lies in [202.4, 247.4] s. The result is a statement about the time limits of our protocol more than about the rewrites, and we draw no conclusion on scalability from it.

D. Budget

Fig. 14 varies the budget M of the selection procedure.

A budget of one already yields most of the benefit: 18 of the final 22 improvements on Dev and 20 of 24 on the held-out statements. The results do not change from $M = 8$ on Dev and from $M = 4$ on the held-out statements. The reason is the size of the admitted set: a held-out statement has 0.89 admitted candidates on average, so a larger budget has nothing to explore. The budget runs execute after the replay arms, whose full timings are in the cache and act as free probes; with $M = 16$ this yields 24 improved held-out statements, one more than the 23 of the first pass in Table II.

TABLE III
SELECTION POLICIES ON THE SAME ADMITTED CANDIDATES. MEAN LATENCY IN SECONDS AS IN (9); F: FASTER THAN THE ORIGINAL; S: SLOWER; U: UNDETERMINED.

	Dev (30)				Held-out (131)			
Policy	Mean	F	S	U	Mean	F	S	U
Full policy	62.3	22	0	0	82.8	24	0	0
Class A only	108.6	9	0	0	83.3	19	0	0
Class B only	63.6	22	0	0	87.0	21	0	0
No hint or join action	85.4	16	0	0	83.3	20	0	0
Generation order	67.6	22	0	0	[82.8, 85.1]	24	0	1
Planner cost only	[102.1, 120.8]	13	0	2	[51.3, 134.4]	19	0	42

E. Policy Variants and Sources

Table III changes one element of the selection policy at a time on the same admitted candidates. These variants are selections from the logged pool; they are not separate runs of the generators.

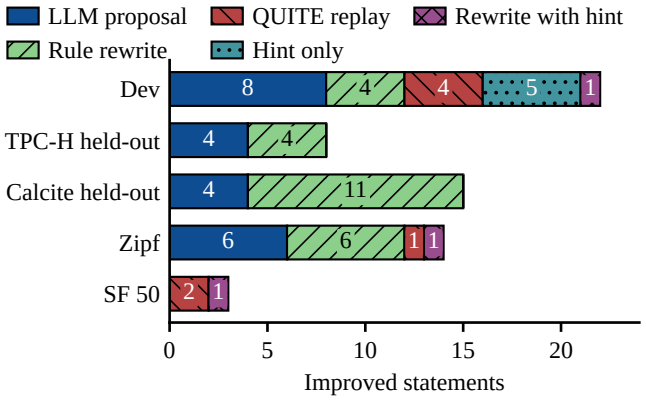


Fig. 15. Source of the rewrite that VeriProbe returns for each improved statement; the numbers are statement counts. A rule rewrite is the output of Calcite rules, including replayed R-Bot rewrites; a hint only candidate keeps the original text.

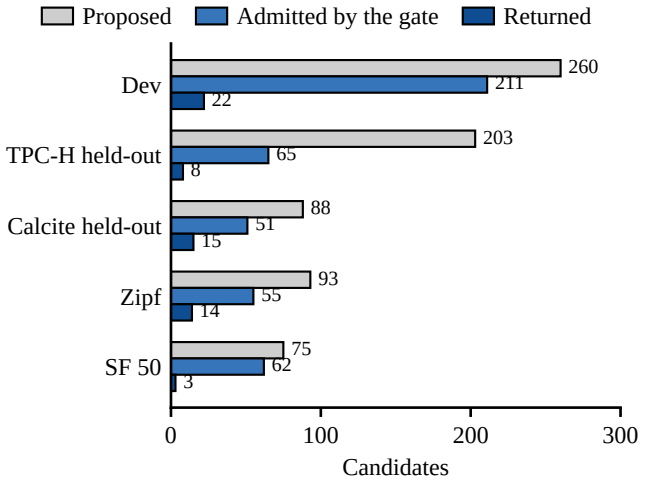


Fig. 16. Candidates per cohort: proposed by the sources, admitted by the gate, and returned by VeriProbe.

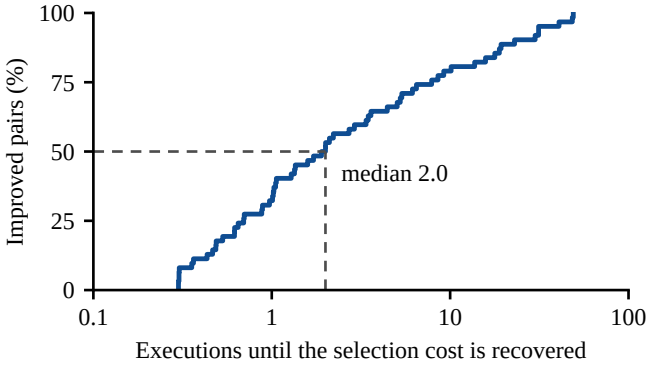


Fig. 17. Cumulative share of improved pairs over $n(q)$ of (7), the number of executions after which the saving covers the cost bound of (6).

Probe and fallback. The last row removes steps 4 and 5: it returns the admitted candidate with the lowest planner cost. On DEV this policy is robustly worse than the full policy, [102.1, 120.8] s against 62.3 s. On the held-out statements the logs decide little: 19 of its selections are faster than the original, 0 are slower, and 42 are undetermined, because the campaign does not time them but reuses probes that are cut off. Its mean lies in [51.3, 134.4] s, an interval that contains the means of Original and of the full policy. The replay arms also select without probe and without fallback, and Section VIII-G times their selections with one cutoff.

Candidate classes. On DEV, class B candidates carry the result: class B alone reaches 63.6 s, class A alone 108.6 s. On the held-out statements the picture reverses: class A alone reaches 83.3 s, which is 0.5 s above the full policy, and class B alone reaches 87.0 s. Fig. 15 shows the same reversal per source. The class B candidates that matter on DEV are the ones that we add there; the frozen LLM proposals alone produce 8 of the 23 held-out improvements. The honest reading is that the gain of VeriProbe on unseen statements comes mostly from selecting rule rewrites safely, and that LLM proposals add a smaller part.

Priority order and planner hints. Replacing the order of (2) by the order of generation costs 5.3 s of mean latency on DEV; on the held-out statements the mean stays in [82.8, 85.1] s, with one run cut off. The order decides which candidates are probed, so it matters only when pools are deep. The fourth row removes every candidate that contains a planner hint or whose action targets a join or a materialization, and it uses the order of generation. The mean on DEV rises to 85.4 s, and 5 of the improvements on DEV are hints on the unchanged statement text. These hints belong to the manually tuned part of the pool; no held-out improvement uses a hint.

F. Evidence and Cost of Selection

Evidence. Over all 211 pairs of database and statement, VeriProbe returns 62 rewrites. 5 carry the label *syntactic*, 2 the label *formal* through the structural check, and 55 the label *bounded*. SQLSolver contributes no admitted candidate in these campaigns. The gate passes only a narrow fragment to it and allows it one second per pair, and the log does not separate these two causes. Only 11.3 % of the selected rewrites are thus backed by evidence for all instances. All 23 held-out improvements are *bounded*. The gate is selective: it rejects 38.2 % of the 719 candidates, and 68.0 % on TPC-H held-out. Fig. 16 shows this funnel per cohort; the campaign log stores admitted candidates only, so we cannot break the rejections down by reason. For the DBA of our scenario, a *bounded* rewrite is a suggestion that has survived six test databases, not a proven replacement.

Cost. We bound the executions that selection adds by (6) and relate them to the saving per execution by (7), counting an original that is cut off at its cutoff. For the 62 improved pairs the median of $n(q)$ is 2.0 executions and the 90th percentile is 22.6; 41 pairs recover the cost within five executions, and the largest value is 49.0. Fig. 17 shows the full distribution.

For the statements that VeriProbe does not improve, the cost

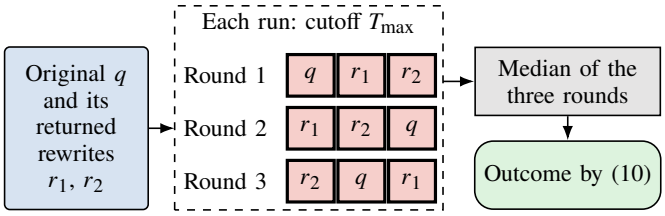


Fig. 18. Order of the runs in the validation session for a pair with two returned rewrites. Steps with a heavy outline execute on the target database.

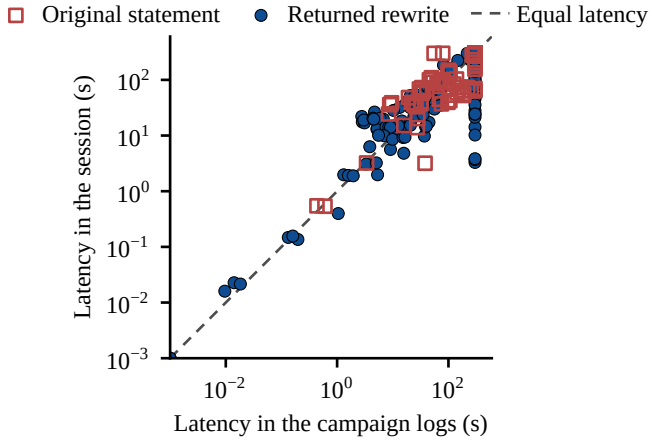


Fig. 19. Latency of each statement in the validation session against its latency in the campaign logs. A run that is cut off is drawn at $T_{\max}$.

is spent without return; on TPC-H held-out this is the common case. The bound excludes the gate, whose runs on the small databases the campaign does not time.

G. Validation Session with Equal Cutoffs

The campaigns leave two doubts open: the arms run with different cutoffs, and VeriProbe selects on the runs that we report. A separate session addresses both.

Protocol. For each pair of database and statement for which at least one arm returns a rewrite, the session times the original and every returned rewrite with the same cutoff $T_{\max}$. It runs three rounds (Fig. 18). Each round executes every statement of the pair once, and the order rotates by one position per round, so that load on the host affects all statements of a pair alike. A statement that reaches the cutoff does not run again. The latency of a statement is the median of its three rounds, where a round that is cut off counts as at least $T_{\max}$; a statement that is cut off in its first round has no other round and is reported as cut off. The session selects nothing. It covers all 88 pairs, and Table IV lists the covered pairs per cohort.

Noise. Other jobs load the workstation during the session. The slowest of the three rounds of a statement takes 1.28 times as long as the fastest at the median and 2.06 times at the 90th percentile. This spread exceeds the tolerance θ, so a verdict close to the tolerance is not reliable. We call a verdict *separated* when all three rounds of the faster statement finish and each of them is shorter than every round of the other statement, where a round that is cut off counts as $T_{\max}$.

Returned rewrites. The session times 62 rewrites that VeriProbe returns in the campaigns. It finds 54 of them again faster than the original (48 separated), 4 within 10 % of the original, 3 slower, and 1 undetermined.

Replay arms. On the covered pairs the campaign logs leave 40 comparisons of a replayed rewrite with its original undetermined. With one cutoff, the session finds 16 of these rewrites faster than the original, 6 within 10 %, 8 slower, and 10 still undetermined. On the held-out pairs it finds 16 of the 18 R-Bot rewrites faster than the original (16 separated) and 1 slower (1 separated); of the 16 QUITE rewrites it finds 3 faster (2 separated) and 6 slower (1 separated).

Probing against planner cost. On the 42 held-out pairs the two selection principles miss different statements. On 6 pairs VeriProbe keeps an original that is cut off, although a replayed rewrite finishes, in 14.2 s to 199.3 s. Each of these rewrites is in the pool of VeriProbe and passes the gate. In the campaign, VeriProbe probes 4 of them and the probe is cut off, 1 inherits the cut off probe of a candidate with the same estimated plan, and 1 is never probed, because candidates with lower planner cost use both probes. In the session, 3 of the rewrites take longer than the probe cap of 45 s and 3 finish below it, so the cap alone does not explain the loss; a probe is a single run without warmup, and the log does not record how long a probe runs before it is cut off. Conversely, on 11 pairs VeriProbe returns a faster rewrite while neither replay arm does: 6 of these rewrites are rule rewrites that the cost filter of R-Bot replay rejects, and 5 are LLM proposals. The first effect dominates the mean, because it concerns originals that do not finish: the mean latency on these pairs is [157.9, 162.5] s for Original, [74.8, 82.2] s for R-Bot replay, 151.5 s for QUITE replay, and [94.9, 99.5] s for VeriProbe. Keeping the fastest timed statement of each pair gives [62.4, 68.1] s. On the held-out statements R-Bot replay therefore outperforms VeriProbe, and the probe step is the cause.

Timing level. Absolute latencies differ between the session and the campaign logs, and the direction depends on the cohort (Fig. 19). For the 13 covered originals of Dev that finish in both, the latency in the session is 0.08 to 1.67 times the latency in the logs, with a median factor of 0.43. For the 25 held-out originals the factor is 0.95 to 4.08, with a median of 1.51. The session finishes 7 of the 38 covered originals that the logs record as cut off, and it cuts off 2 of the 50 that finish in the logs. A latency in this paper therefore holds for the session that measures it, and we compare statements only within one session.

IX. Limitations and Threats to Validity

Replays are not the baselines themselves. The replay arms apply published rewrites on our server and under our gate. A fresh run of either system can produce other rewrites, so our results describe the published rewrites, not the systems.

Selection and report use the same runs. The fallback compares the three timed runs that we also report. That

TABLE IV

VALIDATION SESSION WITH ONE CUTOFF FOR ALL STATEMENTS. MEAN: MEAN LATENCY IN SECONDS OVER THE COVERED PAIRS, WHERE AN ARM WITHOUT A REWRITE COUNTS WITH THE ORIGINAL AND AN INTERVAL ARISES WHEN A STATEMENT IS CUT OFF AFTER A ROUND THAT FINISHES. REWR.: REWRITES TIMED; F: FASTER THAN THE ORIGINAL; W: WITHIN 10 %; S: SLOWER; U: UNDETERMINED.

			R-Bot replay						QUITE replay						VeriProbe					
Cohort	Pairs	Original	Mean	Rewr.	F	W	S	U	Mean	Rewr.	F	W	S	U	Mean	Rewr.	F	W	S	U
DEV	23 of 23	[132.7, 136.0]	88.6	15	12	1	1	1	[100.5, 107.5]	12	9	1	1	1	[67.2, 70.6]	22	16	4	2	0
TPC-H held-out	23 of 23	[147.6, 156.0]	[100.8, 114.3]	6	5	0	0	1	159.1	11	1	3	5	2	[120.4, 128.8]	8	7	0	1	0
CALCITE held-out	19 of 19	170.3	43.2	12	11	0	1	0	142.4	5	2	1	1	1	64.1	15	15	0	0	0
ZIPF	16 of 16	145.4	59.2	12	9	2	1	0	92.6	7	6	0	1	0	23.5	14	14	0	0	0
SF 50	7 of 7	264.3	223.4	2	1	1	0	0	262.1	6	1	0	0	5	220.2	3	2	0	0	1

no selected rewrite is slower in the campaigns is therefore a property of (5) and not an empirical finding, and the reported latency of a selected rewrite is slightly optimistic. The validation session of Section VIII-G removes this dependence for all 88 pairs.

The development cohort is assembled. Its timings come from several campaigns and its pool is tuned on it (Section VIII-A). On the 23 of its pairs that the validation session covers, the mean latency of the original is [132.7, 136.0] s in the session against 199.6 s in the logs. No claim of this paper rests on this cohort alone.

Bounded evidence. 55 of 62 selected rewrites are not proven equivalent. The gate never compares results on the target database.

Server and time limits. The server has little memory and no parallelism, so originals are slow and 30 of 131 held-out originals do not finish. On a tuned server fewer statements leave room for improvement. The probe is the weakest step of the layer: it is a single run without warmup under a fixed cap, and it rejects rewrites that finish while their originals do not (Section VIII-G). A probe cutoff equal to that of the original removes this loss at the price of longer probes; we do not evaluate it.

Coverage. We evaluate 10 of 232 DSB statements; no campaign covers the other 222. The campaign on the 58 Calcite statements of QUITE stops on a schema file that the gate cannot match to the database catalog. The held-out result covers TPC-H and Calcite only.

No search, one model. The priority order is a fixed heuristic, and the LLM candidates are frozen proposals of one model from one generation pass; another model, or repeated sampling, changes the pool. The code base contains a Monte Carlo tree search controller [24] that generates candidates during search; we do not evaluate it, and none of our results speaks about tree search.

X. RELATED WORK

Rule-based rewriting. Rule sets inside optimizers date back to Starburst and Cascades [1], [2], and Calcite makes a large rule set reusable [4]. A learned system explores the order of rule application with Monte Carlo tree search [5]; WeTune discovers and verifies new rules [25], SlabCity synthesizes whole statements [23], and QueryBooster lets users supply rules [26]. VeriProbe consumes the output of such systems as class A candidates.

LLM-based rewriting. LLM-R2 lets an LLM recommend Calcite rules [6], R-Bot adds retrieved evidence and stepwise reflection [7], and GenRewrite and QUITE let the model write the statement [8], [9]. Closest to our work, the tool of Lee et al. for Microsoft SQL Server [11] and ReSequel [12] combine LLM rewriting with verification and execution, each for its own generator. VeriProbe is a layer over several generators, including the published output of other systems; it anchors every decision on the original statement, labels the evidence of what it returns, and reports results as intervals. We make no claim that its verification is stronger than theirs.

Equivalence checking. Provers for SQL equivalence include Cosette [13], EQUITAS [14], SPES [15], SQLSolver [16], and QED [17]; VeriEQL checks equivalence up to a bound [18]. The gate uses SQLSolver for a fragment and bounded testing otherwise; SQLSolver contributes no admitted candidate in our campaigns (Section VIII-F), which is a limit of our integration.

XI. CONCLUSION

For recurring analytical statements on PostgreSQL we place one selection layer on top of several rewrite generators. The layer admits a candidate only through an equivalence gate, probes at most two candidates on the target database, and falls back to the original statement. On 131 held-out statements it improves 23 and lowers mean latency by 18.8 %, and a session with one cutoff finds 22 of 23 returned rewrites faster again. The layer is better than a replay of published QUITE rewrites. It is not better than a replay of published R-Bot rewrites: its single capped probe rejects rewrites that finish while their originals do not, although it also finds rewrites that the planner cost rejects. Most of its selected rewrites rest on bounded testing. Three steps follow directly from these limits: a probe cutoff equal to that of the original, provers with wider coverage, and fresh runs of the compared systems. Reporting cut off runs as intervals, and timing all arms again with one cutoff, are independent of our system; we recommend both for any comparison of rewriters whose time limits differ.

ACKNOWLEDGMENT

Disclosure of content generated by artificial intelligence (AI). The candidate rewrites that Section IV calls LLM pro-

posals are generated by the model gpt-6.1-sol. The text of Sections I to XI, all figures and tables, and the analysis and plotting scripts behind them are drafted with AI coding assistants (Claude and Gemini, used through the Cursor editor) from the campaign logs and reviewed by the author. The experiment code is developed with AI coding assistants.

References

[1] H. Pirahesh, J. M. Hellerstein, and W. Hasan, "Extensible/rule based query rewrite optimization in Starburst," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1992, pp. 39–48.

[2] G. Graefe, "The Cascades framework for query optimization," *IEEE Data Eng. Bull.*, vol. 18, no. 3, pp. 19–29, 1995.

[3] S. Chaudhuri, "An overview of query optimization in relational systems," in *Proc. ACM SIGACT-SIGMOD-SIGART Symp. Princ. Database Syst.*, 1998, pp. 34–43.

[4] E. Begoli, J. Camacho-Rodríguez, J. Hyde, M. J. Mior, and D. Lemire, "Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2018, pp. 221–230.

[5] X. Zhou, G. Li, C. Chai, and J. Feng, "A learned query rewrite system using Monte Carlo tree search," *Proc. VLDB Endow.*, vol. 15, no. 1, pp. 46–58, 2021.

[6] Z. Li, H. Yuan, H. Wang, G. Cong, and L. Bing, "LLM-R2: A large language model enhanced rule-based rewrite system for boosting query efficiency," *Proc. VLDB Endow.*, vol. 18, no. 1, pp. 53–65, 2024.

[7] Z. Sun, X. Zhou, G. Li, X. Yu, J. Feng, and Y. Zhang, "R-Bot: An LLM-based query rewrite system," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5031–5044, 2025.

[8] J. Liu and B. Mozafari, "GenRewrite: Query rewriting via large language models," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 1–26, 2026.

[9] Y. Song, H. Yan, J. Lao, Y. Wang, Y. Li, Y. Zhou, J. Wang, and M. Tang, "QUITE: A query rewrite system beyond rules with LLM agents," arXiv:2506.07675, 2025.

[10] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015.

[11] K. Lee, A. Dutt, V. Narasayya, and S. Chaudhuri, "Verified LLM-based query rewriting for Microsoft SQL Server," *Proc. VLDB Endow.*, vol. 19, no. 12, pp. 4594–4597, 2026.

[12] S. Fathollahzadeh, E. Mansour, and M. Boehm, "ReSequel: Robust LLM-assisted query rewriting and optimization using templatization and sampling," *Proc. VLDB Endow.*, vol. 19, no. 10, pp. 2880–2893, 2026.

[13] S. Chu, C. Wang, K. Weitz, and A. Cheung, "Cosette: An automated prover for SQL," in *Proc. Conf. Innov. Data Syst. Res.*, 2017.

[14] Q. Zhou, J. Arulraj, S. B. Navathe, W. Harris, and D. Xu, "Automated verification of query equivalence using satisfiability modulo theories," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1276–1288, 2019.

[15] Q. Zhou, J. Arulraj, S. B. Navathe, W. Harris, and J. Wu, "SPES: A symbolic approach to proving query equivalence under bag semantics," in *Proc. IEEE Int. Conf. Data Eng.*, 2022, pp. 2735–2748.

[16] H. Ding, Z. Wang, Y. Yang, D. Zhang, Z. Xu, H. Chen, R. Piskac, and J. Li, "Proving query equivalence using linear integer arithmetic," *Proc. ACM Manag. Data*, vol. 1, no. 4, pp. 227:1–227:26, 2023.

[17] S. Wang, S. Pan, and A. Cheung, "QED: A powerful query equivalence decider for SQL," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3602–3614, 2024.

[18] Y. He, P. Zhao, X. Wang, and Y. Wang, "VeriEQL: Bounded equivalence verification for complex SQL queries with integrity constraints," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA1, pp. 1071–1099, 2024.

[19] Nippon Telegraph and Telephone Corporation, "pg_hint_plan," release REL14_1_4_3. [Online]. Available: https://github.com/ossc-db/pg_hint_plan

[20] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2021, pp. 1275–1288.

[21] Transaction Processing Performance Council, "TPC Benchmark H (decision support) standard specification, revision 3.0.1." [Online]. Available: https://www.tpc.org/tpch/

[22] B. Ding, S. Chaudhuri, J. Gehrke, and V. Narasayya, "DSB: A decision support benchmark for workload-driven and traditional database systems," *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3376–3388, 2021.

[23] R. Dong, J. Liu, Y. Zhu, C. Yan, B. Mozafari, and X. Wang, "SlabCity: Whole-query optimization using program synthesis," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 3151–3164, 2023.

[24] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. Eur. Conf. Mach. Learn.*, 2006, pp. 282–293.

[25] Z. Wang, Z. Zhou, Y. Yang, H. Ding, G. Hu, D. Ding, C. Tang, H. Chen, and J. Li, "WeTune: Automatic discovery and verification of query rewrite rules," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2022, pp. 94–107.

[26] Q. Bai, S. Alsudais, and C. Li, "QueryBooster: Improving SQL performance using middleware services for human-centered query rewriting," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 2911–2924, 2023.